TRINITY WESTERN UNIVERSITY

# CMPT 461 Immersive Computing

Assignment #6: VR Interaction Fundamentals (C# Scripting & Physics)

Due Date: March 2nd, 2026 (by 11:59 PM)

**Purpose:**
Students build the core interaction mechanics they will reuse in their team VR application.

🎯 **Learning Outcomes**
Students will learn to:

- Create and attach C# scripts in Unity
- Use object references safely
- Configure colliders, rigidbodies, triggers
- Implement basic XR interactions (grab, button, socket, teleport)
- Build reusable prefabs

**Assignment Description**

📚 Part 0 — XR Interaction Toolkit Setup (Unity 6, works reliably)

**Tutorial**
https://www.youtube.com/watch?v=rKDuYzS-D2E
https://www.youtube.com/watch?v=zbqHNwDpi6Y

---

**Part 0.1 — Create the Project**

1. Open **Unity Hub**
2. **New project → 3D (Built-in)** (URP is also OK)
3. Name: VR_Interaction_Assignment

---

**Part 0.2 — Install the Required Packages**
Open **Window → Package Management → Package Manager**
In the top-left "Packages" dropdown choose **Unity Registry** (or **Packages: Unity Registry**).
Install:

- **XR Interaction Toolkit**
- **XR Plug-in Management**
- **Input System** (if not already installed)

If Unity prompts about input handling, accept switching to the **Input System** (or **Both**) and let Unity restart.

---

**Part 0.3 — Enable OpenXR (Standalone)**

1. Go to **Edit → Project Settings → XR Plug-in Management**
2. In the **PC, Mac & Linux Standalone** tab:

    - Enable **OpenXR**
3. (Recommended) In the same area, check **Project Validation** and fix any warnings Unity lists.

**OpenXR controller profiles (only if you're deploying to real headsets/controllers)**
In **Project Settings → XR Plug-in Management → OpenXR**, enable interaction profiles you care about (e.g., Oculus Touch / Index / WMR). (This area is still correct in modern OpenXR workflows; exact profile names may differ slightly by plugin version.)

---

✅ **The key fix: Import Starter Assets and use the prefab (avoids missing menu options)**

**Part 0.4 — Import XR Interaction Toolkit Samples (Starter Assets + Simulator)**

1. **Window → Project Settings → Package Manager**
2. Select **XR Interaction Toolkit**
3. Go to the **Samples** section
4. Click **Import** for:
    - **Starter Assets** (required)
    - **XR Device Simulator** (or the simulator sample your version offers)

Why this matters: the Starter Assets include a **preconfigured XR Origin prefab** with interactors + locomotion + controller visuals already wired.

---

**Part 0.5 — Build the Scene from Prefabs (most reliable path)**

**0.5a Delete the old camera**
In a new scene, delete **Main Camera** (and any default player controller).

**0.5b Add XR Interaction Manager (one click)**

- Use **GameObject → XR → Interaction Manager**
  (If you don't see the XR menu, don't panic—continue with the prefab method below; many rigs include what you need.)

**0.5c Add the prebuilt XR Rig prefab (this avoids "missing XR Origin (Action-based)")**
In the **Project** window, go to:
Assets/Samples/XR Interaction Toolkit/<version>/Starter Assets/Prefabs/
Drag **XR Origin (XR Rig)** into the **Hierarchy**.

**What you should get automatically** (inside the prefab):

- XR Origin + Camera
- Left/Right controller objects with interactors
- Teleport setup + locomotion setup
- Controller visuals (generic model)

This step is the main reason your old Part 0 broke: Unity/package versions change Create-menu entries, but the **Starter Assets prefab path is stable**.

---

**Part 0.6 — Make Teleport Work (Floor + Teleport layer)**

**0.6a Name the Teleport interaction layer (required by the sample)**

1. Go to **Edit → Project Settings → XR Interaction Toolkit**
2. Find **Interaction Layer Settings**

3. Ensure **Layer 31** is named: Teleport

**0.6b Create a Floor and make it teleportable**

1. **GameObject → 3D Object → Plane**
2. Rename to Floor
3. Add component: **Teleportation Area**
4. Ensure it has a collider (Plane does by default)

---

**Laptop testing (no headset)**

**Part 0.7 — Add a Simulator (Keyboard/Mouse)**

If you imported the simulator sample in **0.4**, you can add it to the scene.

1. In Project window, locate the imported sample folder (commonly under):
   - Assets/Samples/XR Interaction Toolkit/<version>/…
2. Drag **XR Device Simulator** prefab into the **Hierarchy**

Note: Unity documentation indicates a newer **XR Interaction Simulator** exists in newer toolkit versions, and the docs provide install guidance. If your package shows that option, use it; otherwise XR Device Simulator is fine.

---

**Part 0.8 — Quick "Does it work?" Checklist**

In **Hierarchy**, you should have:

- **XR Origin (XR Rig)** (from Starter Assets)
- **XR Interaction Manager** (recommended)
- **XR Device Simulator** (for laptop testing)
- **Floor** (with Teleportation Area)

In **Project Settings**:

- **OpenXR enabled** for Standalone
- XR Interaction Toolkit **Layer 31 named "Teleport"**

In **Play Mode**:

- You should see a headset/camera view
- You should see controller visuals (because the rig prefab includes them)
- Teleport ray + teleport should work on the Floor (Teleportation Area + Teleport layer)

---

✅ **Deliverable (Part 0)**

1. Screenshot of:
   - Project Settings showing **OpenXR enabled**
   - Scene Hierarchy showing **XR Origin (XR Rig)** + **Simulator** + **Floor**
2. Short demo video showing:
   - Controller visuals present
   - Teleport working

---

**Common failure points (Unity 6 / XRI 3.x)**

- **Didn't import Starter Assets** → missing input actions / rig / teleport setup

- **Teleport layer not named "Teleport"** → teleport ray won't validate targets
- **No simulator in scene** (laptop testing) → nothing responds to mouse/keyboard

---

## 📚 Part A — C# Foundations (Unity 6-safe, XR-independent) Tutorial

https://www.youtube.com/watch?v=pIDvwImZvW8

Part A does **not** require XR. Students can do this in a normal Unity scene.

---

**Part A Setup (important for Unity 6 + Input System projects)**

Because Part 0 likely switched your project to the **Input System**, avoid Input.GetKeyDown for Part A. Instead, use the **new Input System** with a tiny script that reads keys directly.

**A0.1 Create a "KeyboardActions" script**

1. Create folder: Assets/Scripts
2. Create script: KeyboardActions.cs
3. Paste this:

```
using UnityEngine;
using UnityEngine.InputSystem;

public class KeyboardActions : MonoBehaviour
{
   [Header("Optional target")]
   public GameObject target;

   [Header("Materials (optional for A2)")]
   public Renderer rend;
   public Material materialA;
   public Material materialB;

   [Header("Timer (optional for A3)")]
   public float delay = 3f;
   public GameObject targetToEnable;

   bool usingA = true;
   float timer = 0f;
   bool fired = false;

   void Awake()
   {
      if (rend == null && target != null) rend = target.GetComponent<Renderer>();
      if (rend == null) rend = GetComponent<Renderer>();
      if (rend != null && materialA == null) materialA = rend.sharedMaterial;
```

```
    }

    void Update()
    {
        // A1 Toggle object on Space
        if (target != null && Keyboard.current.spaceKey.wasPressedThisFrame)
            target.SetActive(!target.activeSelf);

        // A2 Swap material on C
        if (rend != null && materialA != null && materialB != null &&
Keyboard.current.cKey.wasPressedThisFrame)
        {
            usingA = !usingA;
            rend.material = usingA ? materialA : materialB;
        }

        // A3 Timer enable once
        if (!fired && targetToEnable != null)
        {
            timer += Time.deltaTime;
            if (timer >= delay)
            {
                fired = true;
                targetToEnable.SetActive(true);
            }
        }
    }
}
```

This works in Unity 6 with Input System enabled because it uses UnityEngine.InputSystem.

---

**Task A1 — Toggle Object (Space)**

1. Create **Cube** → rename ToggleCube
2. Create **Empty** → rename Manager
3. Add KeyboardActions to Manager
4. Drag ToggleCube into KeyboardActions.target
5. Press Play → press **Space** → cube toggles

---

**Task A2 — Material Swap (C)**

1. Create 2 Materials: RedMat, BlueMat (set colors in Inspector)
2. Use a cube (same or new) with a **Renderer**
3. On the Manager's KeyboardActions:
    - Set **Rend** = cube's Renderer (or drag the cube into target so it auto-finds)

    - Assign **Material A** = RedMat
    - Assign **Material B** = BlueMat
4. Play → press **C** → material swaps

---

**Task A3 — Timer Enable (3 seconds)**

1. Create an object to appear later (e.g., **Sphere**) → rename TimedSphere
2. Disable it (uncheck active box in Inspector)
3. In KeyboardActions:
    - Set **Delay** = 3
    - Set **Target To Enable** = TimedSphere
4. Play → sphere appears after ~3 seconds

✅ **Deliverable Part A:** short demo video showing A1, A2, A3 working.

---

📚 **Part B — Physics & Events (Step-by-Step)**

Tutorials
https://www.youtube.com/watch?v=C_bOx2WEORM
https://www.youtube.com/watch?v=Au8w_6qZxT8

---

**B0 — Create a new scene**

Create a new scene: PartB_PhysicsPlayground.unity

---

**Task B1 — Trigger Zone Event (non-XR, reliable)**

**Goal:** "Player" enters trigger → light toggles.

1. Create **Plane** (floor)
2. Create **Capsule** → rename Player
3. Add **Rigidbody** to Player
    - **Is Kinematic = true**
    - **Use Gravity = false**
4. Tag Player:
    - Inspector → Tag → **Player** (create if needed)
5. Create **Cube** → rename TriggerZone
    - Scale to something like (2, 1, 2)
6. On TriggerZone:
    - Ensure it has a **Box Collider**
    - Check **Is Trigger**
7. Create **Light** (Directional or Point) → rename ToggleLight
8. Create script TriggerEvent.cs and attach to TriggerZone:

```
using UnityEngine;

public class TriggerEvent : MonoBehaviour
{
```

```
    public Light lightToToggle;

    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player") && lightToToggle != null)
            lightToToggle.enabled = !lightToToggle.enabled;
    }
}
```

9. Assign ToggleLight to lightToToggle
10. Move Player into TriggerZone while in Play mode (use Move tool)

✅ Light toggles when Player enters.

---

**Task B2 — Throwable Object (XR Grab, works with Part 0 rig + simulator)**
**Goal:** Grab and throw with XR simulator.
**Scene requirements (must already exist, from Part 0):**

- XR Origin (XR Rig) from Starter Assets
- XR Interaction Manager
- XR Device Simulator (or XR Interaction Simulator)

**Steps**

1. Create **Sphere** → rename Ball
2. Add/verify components:
    - **Sphere Collider** (default) → **Is Trigger OFF**
    - **Rigidbody**
        - Use Gravity: ON
        - Collision Detection: **Continuous** (or Continuous Dynamic)
    - **XR Grab Interactable**
3. XR Grab Interactable settings:
    - **Movement Type**: Velocity Tracking (good for throwing)

**Test**

- Play mode
- Use simulator to move hands/controllers
- Grab Ball and release → it should fly and fall

If it won't grab, check:

- XR Interaction Manager is in scene
- Ball has **Rigidbody + Collider**
- Hands/interactors exist under the XR Rig (Starter Assets rig includes them)

---

**Task B3 — Collision Feedback (material flash)**

1. Create **Cube** → rename Wall
    - Scale it large, place in front of Ball
    - Collider stays default

2. Create material: HitMat (bright color)
3. Create script CollisionFeedback.cs and attach to **Ball**:

```
using UnityEngine;

public class CollisionFeedback : MonoBehaviour
{
    public Renderer rend;
    public Material hitMaterial;
    Material normalMaterial;

    void Awake()
    {
        if (rend == null) rend = GetComponent<Renderer>();
        if (rend != null) normalMaterial = rend.material;
    }

    void OnCollisionEnter(Collision collision)
    {
        if (rend != null && hitMaterial != null)
            rend.material = hitMaterial;
    }
}
```

4. Assign HitMat

✅ When Ball hits Wall/Floor → changes material.

✅ **Deliverable Part B:** screenshots of the scene + demo videos for B1, B2, B3.

---

### 📚 Part C — Core XR Interaction Scene (Main Task — Highly Guided, Unity 6)

Tutorial
https://www.youtube.com/watch?v=PhIyTqgIjfA

---

**C0 — Start from the Part 0 XR scene (recommended)**

Make a new scene: PartC_XRInteractions.unity and include:

- XR Origin (XR Rig) (Starter Assets)
- XR Interaction Manager
- Simulator
- Floor plane

---

**Requirement C1 — Grabbable Object**

1. Create **Cube** → rename Key
2. Add:
    - **Box Collider**

    - **Rigidbody**
    - **XR Grab Interactable**
3. Test grab in Play mode

✅ Works if Part 0 rig is present.

---

**Requirement C2 — Button Interaction (XR Simple Interactable + event)**
**Build a simple "button" that toggles a light**

1. Create **Cube** → rename Button
2. Add component: **XR Simple Interactable**
3. Create **Light** → rename RoomLight
4. Create script ButtonAction.cs on an empty object Logic:

```
using UnityEngine;

public class ButtonAction : MonoBehaviour
{
   public Light targetLight;

   public void ToggleLight()
   {
      if (targetLight != null)
         targetLight.enabled = !targetLight.enabled;
   }
}
```

5. Assign RoomLight into targetLight
6. Wire the event:
    - Select Button
    - In **XR Simple Interactable** find **Activated** (or **Select Entered** if that's what your inspector shows)
    - Add a listener:
        - Drag Logic into the event
        - Choose ButtonAction → ToggleLight()

**How to press it (simulator):**

- Move hand onto Button and "select/activate" with the simulator input (varies by simulator preset, but select events will fire when the interactor selects/presses)

✅ Light toggles.

Note: Depending on XRI version, the easiest reliable event is **Select Entered** (fires when "grab/select" begins). Use that if "Activated" isn't present.

---

**Requirement C3 — Socket Interaction (Key goes into socket)**

1. Create **Empty** → rename Socket
2. Add component: **XR Socket Interactor**

3. Position Socket where you want the key to snap
4. On the Key object:
    - Ensure **XR Grab Interactable** is present
5. Optional: Make socket do something when key is inserted
    - Create script SocketAction.cs on Logic:

```
using UnityEngine;

public class SocketAction : MonoBehaviour
{
   public GameObject doorOrObjectToEnable;

   public void EnableObject()
   {
      if (doorOrObjectToEnable != null)
         doorOrObjectToEnable.SetActive(true);
   }
}
```

6. Create a "Door" cube and set it inactive initially
7. In Socket Interactor events:
    - **Select Entered** → call EnableObject()

✅ Grab Key → move to Socket → it snaps in → door appears.

---

**Requirement C4 — Teleport Area**

1. Create **Plane** named Floor (or reuse existing)
2. Add **Teleportation Area**
3. Make sure your project has the **Teleport** interaction layer set (Starter Assets expects it)
    - Project Settings → XR Interaction Toolkit → Interaction Layers (Layer 31 = Teleport)

✅ Teleport works in Play mode with simulator.

---

**Scene Requirement (simple, guaranteed chain)**

Build this mini experience:

1. Grab Key
2. Insert into Socket
3. "Door" appears or light turns on
4. Teleport around the floor

✅ **Deliverable Part C:** demo video showing C1–C4 + the full interaction chain.

---

**Fast troubleshooting (the 6 things that fix 95% of issues)**

1. **XR Interaction Manager exists** in scene
2. XR Rig is **Starter Assets XR Origin (XR Rig)** (not an empty rig missing interactors)
3. Grabbables have **Rigidbody + Collider + XR Grab Interactable**

4. Trigger events require **Is Trigger ON** + at least one **Rigidbody**
5. Collision events require **Is Trigger OFF** + at least one **Rigidbody**
6. Teleport requires **Teleportation Area** + Teleport layer naming in XR Interaction Toolkit settings

**Marking scheme**:

- Part 0 (10 marks): screenshots of the teleportation area and demo video(s)
- Part A (10 marks): 3 scripts (ToggleObject, ChangeMaterial, and TimerAction) and demo video(s) of Tasks A1, A2, and A3
- Part B (10 marks): Scripts and screenshots of physics playground scene and demo video(s) of Tasks B1, B2, and B3
- Part C (15 marks): demo video(s) of Requirements 1, 2, 3, and 4, and Scene Requirement
- Writing a short reflection on (5 marks):
    - What interaction will be reused in your team project
    - Biggest difficulty

Total marks in Assignment #5: 50 marks.

**Assignment Submission:**

You need to submit the following to Moodle before the due date:

- A document (PDF preferred) with your reflections and screenshots of Part 0 and B
- A zip file of your Unity project **standalone executable** and its necessary folder(s) and files for Parts 0, A, B, and C, including DLLs to run and play your assignment (not your original Unity project)
- Demo video files of screen recordings

**Note:** There are instructions on how to create a Unity project standalone executable in the previous assignment instructions.

When your Unity project **standalone executable** is executed on the computers of the instructor or TA, it should be playable without any errors. The instructor or TA will not fix any errors to evaluate your assignment.

Note: If your file size is greater than 20 MB, upload your files (or a zip file) to an online storage service, such as OneDrive, Google Drive, or Dropbox, and submit the download link. Please make sure that anybody can download from the link.